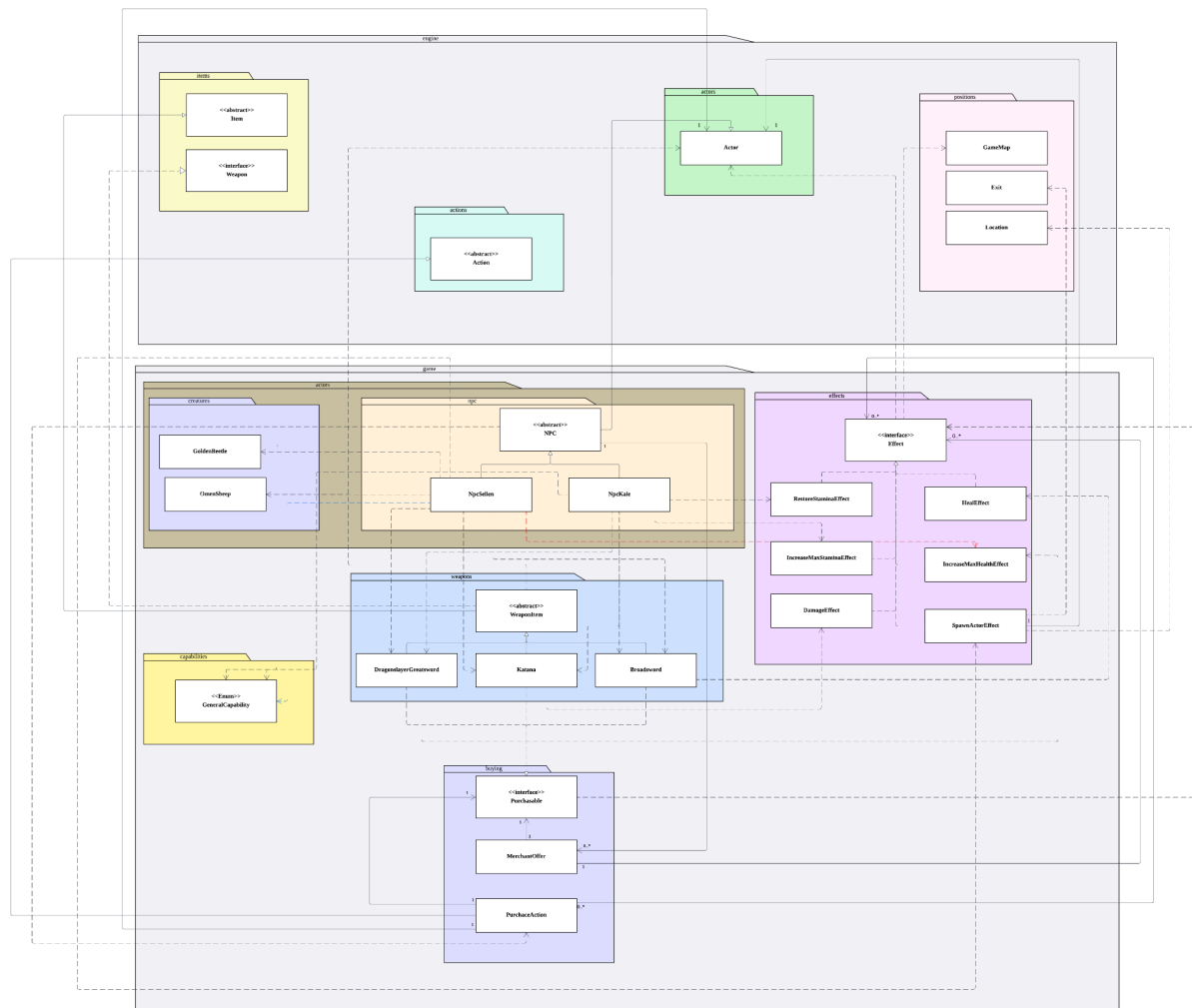


Revised Design Rationale: Item Purchase from Merchants



1. Introduction

This rationale covers the "Farmer" purchasing weapons from merchants. It addresses the purchase process, weapon/merchant-specific effects, and extensibility, ensuring the design is robust and aligns with sound software engineering practices.

2. Design Goals

Single-Menu Purchase: Buying occurs in the main action menu, costing 1 game tick, providing a seamless player experience.

Complex Effects: Implement base weapon effects and merchant-dependent additional effects. (Specifics like Broadsword healing 10 HP, Sellen's Broadsword adding 20 Max HP, Kale's Broadsword adding 30 Max Stamina, etc., are detailed in requirements).

Extensibility: Easily add new merchants, items, and effects with minimal changes to existing systems, adhering to principles like OCP.

3. Design Analysis & Decisions

3.1. Core Purchase & Effect System

Requirement: Manage item purchases with varying costs and layered effects.

Option A: Monolithic Player Class Logic

Description: All purchase-related logic (merchant interaction, item selection, rune deduction, effects) is coded directly within the Player class.

Pros	Cons
Might seem simpler for an extremely limited initial set of items, requiring fewer files.	Violates SRP (Player becomes overly complex, hard to test).
	Violates OCP (modification needed for any new item/merchant/effect). Leads to DRY violations (duplicated logic) and low maintainability.

Option B: Interface-Driven, Compositional Design

Description:

Purchasable Interface: Implemented by items (e.g., Broadsword). Defines `getBasePurchaseEffects()` and `sellTo(Actor actor)`.

Effect Interface: Implemented by effect classes (e.g., HealEffect). Defines `applyEffect(Actor actor, GameMap map)`.

MerchantOffer Class: Holds a Purchasable item, price, and merchant-specific `additionalEffects` (an `ArrayList<Effect>`).

PurchaseAction Class: Takes a MerchantOffer. Manages runes, item transfer, and applies base and additional effects.

Pros	Cons
------	------

Adheres to SRP & OCP, reducing complexity and improving testability of individual components	Slightly more classes initially, which may involve a minor learning curve for the overall interaction pattern.
High maintainability & extensibility.	

Decision & Justification for 3.1: Option B

Chosen for strong SOLID principle adherence and superior maintainability, crucial for evolving game development.

OCP Example: Adding a new "Fire Sword" (implementing Purchasable) with a "Burn" effect (a new Effect class applying damage over time) sold by a new merchant (a new MerchantOffer) requires no changes to PurchaseAction or Player. This isolation of changes significantly reduces regression risks.

Maintainability: If Sellen's Broadsword offer changes (e.g., also grants a small, temporary defense boost), only her MerchantOffer needs updating with a TemporaryDefenseBuffEffect. This localized change is much safer and faster than navigating a large, monolithic Player class.

Requirements Addressed: This design cleanly layers effects. For instance, the Broadsword's inherent HealEffect(10) is applied, then Sellen's MerchantOffer might add an IncreaseMaxHealthEffect(20). Similarly, the Katana's base DamageEffect(25) is applied, followed by Sellen-specific effects like SpawnActorEffect (for an OmenSheep near sellenActor), HealEffect(10), and IncreaseMaxStaminaEffect(20).

3.2. UI Integration & Action Cost

Requirement: Purchases in the main action menu, costing one game tick.

Option A: Dedicated Sub-Menu System

Description: Merchant interaction opens a separate buying UI

Pros	Cons
Potentially clearer for vast inventories if not handled well by the main menu.	Violates "no separate menu" requirement. Inconsistent UX, potentially confusing players. Higher implementation cost.

Option B: Integration into Existing Action System

Description: Merchants' allowableActions method generates PurchaseAction instances. Player selects these from the standard action menu. Engine's Action lifecycle handles tick consumption.

Pros	Cons
Meets requirements directly.	Long action list if merchant has many items (addressable by Menu pagination, an existing or easily added Menu feature).
Consistent UX, making the game feel more cohesive.	
Easy implementation by leveraging existing systems.	
SRP maintained (NPCs offer actions, PurchaseAction executes purchase).	

Decision & Justification for 3.2: Option B

Selected for direct requirement fulfillment, ease of implementation, and UX consistency. This approach treats purchasing as just another player action, fitting naturally into the game's flow.

3.3. Effect Implementation Details

Requirement: Implement specific effects like healing, stat changes, and actor spawning.

Option A: Direct Effect Logic in PurchaseAction

Description: PurchaseAction.execute uses conditionals for all effect logic.

Pros	Cons
Quick for very few, static, and simple effects.	Violates OCP & SRP. Violates DRY. Becomes unmaintainable and error-prone as effects grow in number or complexity.

Option B: Effect Interface and Concrete Classes

Description:

Effect interface: Defines `applyEffect(Actor actor, GameMap map)`.

Concrete classes for each effect: e.g., `HealEffect(int healAmount)`, `IncreaseMaxHealthEffect(int increaseAmount)`, `SpawnActorEffect(Supplier<Actor> actorSupplier, Actor spawnNearActor)`.

`Purchasable` and `MerchantOffer` hold lists of these `Effect` objects. `PurchaseAction` iterates and calls `applyEffect`.

Pros	Cons
OCP & SRP compliant. Promotes DRY. Flexible, compositional, and highly maintainable. Testing individual effects is straightforward.	Results in a larger number of small, focused classes.

Decision & Justification for 3.3: Option B

Chosen for flexibility and SOLID adherence, allowing for a robust and adaptable effects system.

Example: A new `"GrantsTemporaryInvisibilityEffect"` can be a new `Effect` class, taking `duration` as a parameter. It can then be added to special offers or rare item purchases without altering `PurchaseAction` or other unrelated effects. This compositional nature is powerful.

Requirements Addressed: All specified effects are covered. The `SpawnActorEffect` takes an `actorSupplier` (e.g., `GoldenBeetle::new`) and an optional `spawnNearActor` (e.g., `sellActor` for her Katana's Omen Sheep, ensuring the sheep spawns near her as required). Stat changes are encapsulated within effects like `IncreaseMaxHealthEffect`, which call appropriate `Actor` methods (like `modifyAttributeMaximum`) with relevant parameters (like `increaseAmount`, which internally maps to the `value` parameter in the `Actor`'s method).

4. Conclusion

This design, using `Purchasable`, `MerchantOffer`, `PurchaseAction`, and `Effect` components, robustly meets all requirements. It prioritizes SOLID and DRY principles, ensuring a maintainable and extensible system. This clear separation of concerns and use of interfaces allows developers to introduce new items, merchants, or complex purchasing outcomes with confidence and minimal impact on existing, stable code. The comparison of alternatives validates this compositional, interface-driven approach for managing complexity and facilitating long-term evolution of the game's features.